TECHNICAL ENHANCEMENT DOCUMENT

Gantt Resource Panel Loading: Synchronous → Asynchronous Migration via Page Background Tasks

Business Central AL extension — DHTMLX Gantt control add-in
(page 50620 "Gantt Demo DHX 2")

Scope	Day Planning / Resource data loading for the Gantt resource panel
Objects touched	Page 50620, Codeunit 50613, Control Add-in interface + wrapper.js
Objects added	Query 50712 "Day Planning By Job Task", Codeunit 50713 "Gantt BG Resource Panel Data"
Platform feature used	Business Central Page Background Tasks (child-session async compute)
Prepared	2026-08-31

Contents

1. Executive Summary

Page 50620 "Gantt Demo DHX 2" renders a DHTMLX Gantt chart backed by Business Central data. On open, it used to build and send three JSON payloads synchronously in one blocking AL call chain: Job Tasks, Resources, and Day Plannings. The Day Planning table is large (28,025 rows in the environment used for verification), and the resource panel that consumes that data **starts hidden** and is not shown until the user explicitly clicks "Show Resource Panel" or right-clicks a task bar. The synchronous load was therefore paying the full cost of the largest dataset on every single page open, for data the user had not yet asked to see.

This enhancement removes that cost from the critical path using Business Central's **Page Background Task** feature: Job Task data (fast, and needed immediately for the chart to render) stays synchronous; all Day Planning and task-scoped Resource loading is deferred to a read-only child session that runs off the interactive request path, and the result is pushed into the control add-in only once it is ready. Every interaction that used to trigger a synchronous Day Planning fetch — Show Resource Panel, right-click "Show Job Resources", Previous/Today/Next, Refresh Data, and the post-drag resource-panel reload — now goes through the same asynchronous mechanism.

A secondary, related fix targets the actual algorithmic bottleneck inside the Day Planning fetch itself: the two procedures that build resource-panel JSON looped once per Job Task and issued a separate `Record.FindSet()` per iteration (an N+1 round-trip pattern). These were rewritten on top of a single Query object, so a multi-task resource-panel reload now costs one database pass regardless of how many tasks are in scope.

Live testing surfaced an undocumented platform constraint that required a design change beyond the original plan — detailed in Section 7 — and both a correctness cross-check (against live BC data via API queries) and a live-browser behavioral verification were performed; results are in Section 8.

2. Problem Statement

Before this change, `ControlReady` on page 50620 called `LoadAllData()`, which built and sent Job Task, Link, Holiday, Resource, and Day Planning JSON *all synchronously, in one AL call chain*, before the trigger returned control to the browser:

`BEFORE — src/dhx/ganttdemo2/page_50620_GanttDemo.al`

```
trigger ControlReady()
begin
    setup.EnsureUserRecord();
    setup.get(UserId);
    LoadAllData();                // ← blocks until Job Tasks AND Day Plannings are both built
    ResourcePanelFlag := false;    // panel hidden AFTER the expensive fetch already ran
    CurrPage.DHXGanttControl2.SetResourcePanelVisibility(ResourcePanelFlag);
end;
```

Two compounding issues:

- **Wasted work on every open.** `ResourcePanelFlag` was only set to `false` *after* `LoadAllData()` had already run — and inside `LoadAllData`, the Day Planning/Resource fetch was gated on the *pre-reset* panel state (set to `true` in `OnOpenPage`), so the full Day Planning payload for the visible date window was always built and sent to a control that would be hidden one line later.
- **N+1 round trips inside the fetch itself.** When the panel *was* populated (Show Resource Panel, task right-click, post-drag reload), `GetDayPlanningsByJobTaskAsJson` and `GetResourcesByJobTaskAsJson` looped over every Job Task in the requested scope and issued a fresh `DayPlanning.SetRange(...); DayPlanning.FindSet()` per task — one database round trip per task, against a table that is large by design (Day Planning is the finest-grained scheduling entity in this system).

> **OBSERVED IMPACT**
>
> On the environment used for verification (28,025 Day Planning rows), the synchronous path added measurable latency to every Gantt open, Previous/Next click, Today click, and Refresh Data click — regardless of whether the user ever opened the resource panel that session.

3. Solution Architecture

The fix splits the load into a fast synchronous half and a deferred asynchronous half, using BC's **Page Background Task** feature (a read-only child session on the same server instance that returns results to the parent page as a `Dictionary of [Text, Text]`).

3.1 What stays synchronous

Bar/day-off colors, date-range resolution, the filter-toolbar sync, `LoadProject`, Holidays, column visibility, **Job Task JSON** (`LoadTaskData`), Link JSON, and the final `RenderGantt(false)`. These are either cheap or required for the chart to be visible at all, and Job Task loading was explicitly kept direct per the original request.

3.2 What moved to a background task, and the end-to-end flow

1. **User action** (open Gantt, click Show Resource Panel, right-click a task, Previous/Today/Next, Refresh Data, or a post-drag reload) reaches an AL trigger on page 50620.
2. ASYNC STARTS HERE
   Enqueue. The trigger calls `EnqueueFilteredResourcePanelReload` or `EnqueueDefaultResourcePanelReload`, which builds a `Dictionary of [Text, Text]` of scope parameters and calls `CurrPage.EnqueueBackgroundTask(...)`, then returns immediately. The panel is shown (possibly empty) without waiting.
3. **Background child session.** Codeunit 50713 "Gantt BG Resource Panel Data" runs in its own read-only session/transaction, reads its parameters via `Page.GetBackgroundParameters()`, builds the Resources/Day Plannings JSON (via the Query-based codeunit 50613 procedures, Section 5), and calls `Page.SetBackgroundTaskResult(Result)`.
4. **Completion callback.** `OnPageBackgroundTaskCompleted` fires on the page with the result dictionary. It *cannot* call the control add-in directly (Section 7) — it stashes the JSON into plain AL vars and sets a "result pending" flag, then calls a control add-in method to arm a client-side poll.
5. **Client-side delivery.** wrapper.js runs a short, bounded poll (500 ms × up to 60 attempts) calling back into a normal AL trigger, `OnPollResourcePanelResult`, which — from a normal synchronous call stack, where control add-in calls *are* allowed — pushes the pending JSON into `LoadResourcesData` / `LoadDayPlanningsData` and tells JS to stop polling.

At no point does the page's main call chain wait for step 3 or 4 — the enqueueing trigger (step 2) always returns immediately, satisfying the "whole cycle stays asynchronous" requirement that motivated this design.

> DIRECT ANSWER — EXACTLY WHERE DOES THE CYCLE BECOME ASYNCHRONOUS?
>
> It is one specific statement: the call to `CurrPage.EnqueueBackgroundTask(...)`. Everything *before* that statement, in the same procedure, runs synchronously as normal AL. The instant that statement executes, control returns to the caller — Business Central schedules codeunit 50713 to run in a separate child session, and the AL code that called `EnqueueBackgroundTask` does **not** wait for it to finish. There are exactly two places in this codebase where that statement appears:

Call site	File : line (as of this document)	Used by
`EnqueueFilteredResourcePanelReload`	`page_50620_GanttDemo.al : 1423`	Right-click "Show Job Resources"; post-drag reload of a task-scoped panel

`EnqueueDefaultResourcePanelReload`	`page_50620_GanttDemo.al : 1468`	Show Resource Panel; Previous/Today/Next; Refresh Data; initial open (if the panel were left visible)

See Section 4.2 for the exact line highlighted in its surrounding code, and Section 3.4 for a step-by-step trace of what runs before vs. after that line within a single trigger invocation.

3.3 A short primer: parent session, child session, and why parameters are `Dictionary of [Text, Text]`

Every time a user has page 50620 open, Business Central holds a **parent session** for it — the normal client/server connection that runs every trigger you've read on this page so far (`ControlReady`, `OnShowResourcesForTask`, action triggers, and so on). A Page Background Task spins up a completely separate **child session**: its own connection to the same server instance, its own `OpenCompany` call, its own transaction. It is, for practical purposes, as if a second user silently logged in, ran one read-only codeunit, and logged straight back out.

Because the parent and child are genuinely separate sessions (potentially even separate OS threads/processes under the hood), AL cannot simply "pass a Record variable" from one to the other — a `Record "Job Task"` instance, a `Record.Mark()` list, an open `Query` handle, none of that is meaningful outside the session that created it. The platform's answer is to only allow the one data shape that is trivially safe to copy between sessions: a `Dictionary of [Text, Text]` — plain string keys and values, serialized in, deserialized out. That is *why* this enhancement had to invent a text encoding for its inputs and outputs instead of reusing the Record-based shapes the old synchronous code used:

- **Job Task scope** — previously a marked `Record "Job Task"` set; now a `"JobNo|JobTaskNo"` pair per task, joined with `;`, in one Dictionary value (`BuildResourcePanelJobTaskKeysText`, Section 4.2).
- **Dates** — previously native `Date` variables passed by value in a normal procedure call; now formatted as `"YYYY-MM-DD"` text and re-parsed with `Evaluate()` on the far side (codeunit 50713's `EvaluateDateParam`, Section 6).
- **Booleans** — previously native `Boolean` parameters; now the literal text `"true"` / `"false"` — and specifically *not* `Format()`, which is a real pitfall covered in Section 7.4.
- **The JSON results themselves** — already plain `Text`, so no extra encoding was needed for the output side; they go straight into the Result dictionary under the keys `'resourcesJson'` / `'dayPlanningsJson'`.

3.4 Trace of one call, statement by statement

To make the synchronous/asynchronous boundary completely concrete, here is exactly what happens, in order, for a single click of the "Show Resource Panel" action (the `EnqueueDefaultResourcePanelReload` path). Everything up to and including step 4 happens inside the one AL trigger invocation triggered by the click; step 5 onward happens later, kicked off separately by the platform.

1. User clicks "Show Resource Panel". AL trigger starts running — **synchronous**, normal call stack.
2. `EnqueueDefaultResourcePanelReload(...)` builds the `TaskParameters` Dictionary (Mode, LoadResources, LoadDayPlannings, JobFilter, JobTaskFilter, AnchorDate) — still synchronous, no server round trip yet, just building a variable in memory.
3. `CurrPage.EnqueueBackgroundTask(NewTaskId, Codeunit::"Gantt BG Resource Panel Data", TaskParameters, 30000, PageBackgroundTaskErrorLevel::Warning)` **executes.** This is the exact statement from the answer box above. Business Central registers a new child session request and assigns it a TaskId (written into `NewTaskId`) — but it does **not** block waiting for that child session to run.

4. **STILL THE SAME TRIGGER, STILL SYNCHRONOUS** The very next lines run immediately, with no wait: `ResourcePanelTaskId := NewTaskId`, `PendingResultAvailable := false`, then `CurrPage.DHXGanttControl2.NotifyResourcePanelTaskPending()` (arms the JS poll timer). The trigger then returns to the client. From the user's point of view, the panel is already open — just empty so far.

5. **SEPARATE CHILD SESSION, RUNS IN PARALLEL** At some point after step 3 (milliseconds to a couple of seconds later, depending on server load and how much data codeunit 50713 has to read), the child session actually executes codeunit 50713's `OnRun()`, reads the Dictionary, runs the Query, and calls `Page.SetBackgroundTaskResult(...)`.

6. **BACK ON THE PARENT SESSION, BUT A NEW/SEPARATE TRIGGER FIRING** BC invokes `OnPageBackgroundTaskCompleted` on the page — this is a distinct trigger invocation from the one in steps 1–4, running later, asynchronously with respect to it. It stashes the JSON and sets `PendingResultAvailable := true`, then returns (it still cannot touch the control add-in — Section 7).

7. **CLIENT-DRIVEN, UP TO 500MS LATER** wrapper.js's poll timer fires `OnPollResourcePanelResult`, which finds `PendingResultAvailable = true` and finally calls `LoadResourcesData` / `LoadDayPlanningsData` on the control add-in. The panel fills in.

4. Implementation Detail — Page 50620

4.1 ControlReady / LoadAllData

`ResourcePanelFlag` is now set to `false` *before* `LoadAllData()` runs, so its resource-panel dispatch sees the panel is hidden and enqueues nothing on the very first open:

`AFTER — page_50620_GanttDemo.al, ControlReady trigger`

```
trigger ControlReady()
begin
    setup.EnsureUserRecord();
    setup.get(UserId);
    // Panel starts hidden - set this BEFORE LoadAllData() so its resource-panel
    // dispatch (see LoadAllData) sees ResourcePanelFlag = false and doesn't enqueue
    // any background task at all on initial open. Nothing is shown until the user
    // clicks "Show Resource Panel" or a task bar, so there is nothing to fetch yet.
    ResourcePanelFlag := false;
    LoadAllData();
    CurrPage.DHXGanttControl2.SetResourcePanelVisibility(ResourcePanelFlag);
end;
```

Inside `LoadAllData`, the resource-panel section no longer calls the codeunit directly — it only enqueues:

```
// Resource-panel data (resources + Day Plannings) is not fetched synchronously here -
// it isn't even visible until the panel is shown, so building/sending it eagerly on
// every LoadAllData was pure waste. Instead this just kicks off a Page Background Task
// and returns immediately; OnPageBackgroundTaskCompleted pushes the JSON into the
// control add-in once it's ready.
if (ResourcePanelJobNo <> '') and (ResourcePanelJobTaskNo <> '') then
    EnqueueFilteredResourcePanelReload(ResourcePanelFromDate, ResourcePanelToDate)
else
    if ResourcePanelFlag then
        EnqueueDefaultResourcePanelReload(Setup."Load Resources", Setup."Load Day Plannings");
```

This single dispatch point is shared by `ControlReady`, `RefreshGantt` (used by Previous/Today/Next/Refresh Data), and every other caller of `LoadAllData` — so all of them inherit the asynchronous behavior for free.

4.2 Enqueue helper procedures

Two scopes are enqueued, mirroring the two existing synchronous entry points they replace:

```
local procedure EnqueueFilteredResourcePanelReload(pFromDate: Date; pToDate: Date)
var
    TaskParameters: Dictionary of [Text, Text];
    NewTaskId: Integer;
    JobTaskKeysText: Text;
begin
    JobTaskKeysText := BuildResourcePanelJobTaskKeysText();
    if JobTaskKeysText = '' then
        exit;

    TaskParameters.Add('Mode', 'Filtered');
    TaskParameters.Add('LoadResources', 'true');
    TaskParameters.Add('LoadDayPlannings', 'true');
    TaskParameters.Add('JobTaskKeys', JobTaskKeysText);
    TaskParameters.Add('FromDate', Format(pFromDate, 0, '<Year4>-<Month,2>-<Day,2>'));
    TaskParameters.Add('ToDate', Format(pToDate, 0, '<Year4>-<Month,2>-<Day,2>'));

    CurrPage.EnqueueBackgroundTask(NewTaskId, Codeunit::"Gantt BG Resource Panel Data",     // <=== (A) ASYNC
STARTS HERE
        TaskParameters, 30000, PageBackgroundTaskErrorLevel::Warning);                     //     control
returns to the caller now
    ResourcePanelTaskId := NewTaskId;                   // (B) still synchronous - runs immediately, does not
wait for (A)
    PendingResultAvailable := false;                    // (B) any earlier not-yet-delivered result is now
stale
    CurrPage.DHXGanttControl2.NotifyResourcePanelTaskPending(); // (B) arm the poll loop - still the same call
stack as (A)
end;
```

> **READING THE ANNOTATION ABOVE**
>
> Line **(A)** is the exact `page_50620_GanttDemo.al:1423` statement referenced in Section 3.2's answer box — the one line where "asynchronous" actually begins. Lines **(B)** are not asynchronous themselves; they are ordinary AL statements that happen to run *immediately after* an async task was scheduled, on the same call stack, without waiting for that task. The background task's own work (codeunit 50713 running, Section 6) happens later, off this call stack entirely — see the numbered trace in Section 3.4.

`EnqueueDefaultResourcePanelReload` (page_50620_GanttDemo.al:1468 for its own `EnqueueBackgroundTask` call) is the same shape for the unfiltered (all resources / all Day Plannings in the visible period) case, and additionally guards against a genuine `Format(Boolean)` pitfall — see Section 7.4.

Job Tasks are identified by plain `"JobNo|JobTaskNo"` text keys, not `Record.Mark()`, because a background task runs in a different session — marks set on the interactive page's Record instance do not exist there:

```
local procedure BuildResourcePanelJobTaskKeysText(): Text
var
    Keys: Text;
    ChildTaskId: Text;
begin
    if (ResourcePanelJobNo = '') or (ResourcePanelJobTaskNo = '') then
        exit('');

    Keys := ResourcePanelJobNo + '|' + ResourcePanelJobTaskNo;
    foreach ChildTaskId in ResourcePanelChildTaskIds do
        Keys += ';' + ChildTaskId;
    exit(Keys);
end;
```

4.3 New page variables

Variable	Purpose
`ResourcePanelTaskId`	TaskId of the most recently enqueued resource-panel background task. Completion/Error triggers discard any result whose TaskId doesn't match — the guard against a stale result from a superseded reload (e.g. rapid Previous/Next clicks).
`PendingResourcesJson` `PendingDayPlanningsJson`	Set by `OnPageBackgroundTaskCompleted`; read and cleared by `OnPollResourcePanelResult` once actually delivered into the control add-in.
`PendingResultAvailable`	Flag the poll trigger checks before doing any work — avoids re-delivering the same result on every poll tick.

5. Implementation Detail — Query 50712 & Codeunit 50613 (N+1 removal)

> **WHAT "N+1 ROUND TRIPS" MEANS HERE, CONCRETELY**
>
> A "round trip" is one request/response exchange between the AL runtime and the SQL database underneath Business Central. `Record.FindSet()` issues one such round trip and streams back the matching rows. The old code called `DayPlanning.FindSet()` *once per Job Task in scope*, inside a `repeat ... until JobTask.Next() = 0` loop — so a resource panel scoped to, say, 12 child tasks made 12 separate round trips to fetch Day Plannings, then (in `GetResourcesByJobTaskAsJson`) *another* 12 to re-derive the resource list, for 24 round trips total to answer one right-click. This is the textbook "N+1 query problem": 1 query to get the list of parent rows (Job Tasks), then N more queries (one per parent row) to get each one's children, when a single query with the right filter could have returned everything at once. The fix is not "async" by itself — it is a separate, compounding improvement: collapse the N per-task round trips into exactly 1, using a Query object whose filter covers every task's key at once (Section 5.1 below explains how the OR-list filter makes that possible even though a Query cannot iterate a caller-supplied list of keys directly).

5.1 The two procedures feeding the resource panel used to accept a marked `Record "Job Task"` set and loop:

`BEFORE (conceptual shape, codeunit 50613)`

```
repeat
    DayPlanning.Reset();
    DayPlanning.SetRange("Job No.", JobTask."Job No.");
    DayPlanning.SetRange("Job Task No.", JobTask."Job Task No.");
    DayPlanning.SetRange("Plan Date", FromDate, ToDate);
    if DayPlanning.FindSet() then     // ← one round trip PER Job Task
        repeat
            JsonArray.Add(CreateDayPlanningJsonObject(DayPlanning));
        until DayPlanning.Next() = 0;
until JobTask.Next() = 0;
```

They now take a plain `List of [Text]` of `"JobNo|JobTaskNo"` keys (also required so the same procedures are callable from the background task's own session — marks don't survive across sessions) and run against one new Query object, `Query 50712 "Day Planning By Job Task"`:

`AFTER — src/query/query_50712_DayPlanningByJobTask.al`

```
query 50712 "Day Planning By Job Task"
{
    QueryType = Normal;
    elements
    {
        dataitem(Day_Planning; "Day Planning")
        {
            filter(JobNoFilter; "Job No.") { }
            filter(JobTaskNoFilter; "Job Task No.") { }
            filter(PlanDateFilter; "Plan Date") { }

            column(SystemId; SystemId) { }
            column(JobNo; "Job No.") { }
            column(JobTaskNo; "Job Task No.") { }
            column(PlanDate; "Plan Date") { }
            column(DayLineNo; "Day Line No.") { }
            column(StartTimeAssigned; "Start Time Assigned") { }
            column(EndTimeAssigned; "End Time Assigned") { }
            column(StartTimeRequested; "Start Time Requested") { }
            column(EndTimeRequested; "End Time Requested") { }
            column(AssignedHours; "Assigned Hours") { }
            column(RequestedHours; "Requested Hours") { }
            column(NonWorkingMinutesAssigned; "Non Working Minutes Assigned") { }
            column(NonWorkingMinutesRequested; "Non Working Minutes Requested") { }
            column(AssignedResourceNo; "Assigned Resource No.") { }
            column(RequestedResourceNo; "Requested Resource No.") { }
            column(VendorNo; "Vendor No.") { }
            column(PlanStatus; "Plan Status") { }
            column(WorkOrderNo; "Work Order No.") { }
        }
    }
}
```

5.2 Why a Query object specifically — not just a smarter Record filter

Collapsing the loop's N round trips down to one round trip does not, strictly speaking, require a Query object — a single `Record.SetFilter("Job Task No.", 'a|b|c')` (an AL OR-filter across every requested task) followed by one `FindSet()` would already fix the N+1 problem, because it is still exactly one round trip regardless of how many alternatives the OR-filter lists. The Query object is used here on top of that fix, for reasons that matter specifically because this data feeds a JSON payload rather than driving further record edits:

- **Field projection.** A Query's `elements` block declares exactly the columns it returns (17 named columns in Section 5, matching precisely what `BuildDayPlanningJsonObject` needs) — the generated SQL only ever selects those columns. A plain `Record` variable, by contrast, defaults to fetching every field on the table unless the caller remembers to call `SetLoadFields`; the Query bakes that discipline into the object definition itself, so it cannot be forgotten later by a future edit.
- **Read-only by construction.** A Query has no `Insert` / `Modify` / `Delete` — it can only be `Open()` ed and `Read()`. That matches the background task's own constraint exactly (Section 6: the child session's database access is read-only and errors on any write), so using a Query here makes the read-only intent explicit in the type itself rather than relying on nobody accidentally calling `DayPlanning.Modify()` somewhere in the loop.
- **Natural fit for grouping/distinct work.** `GetResourcesByJobTaskAsJson` needs the *distinct* set of resource numbers referenced across many Day Planning rows. Query objects are the standard AL tool for that shape of problem (they support `Method = Totals` /grouping natively); this implementation currently does the distinct-collection in AL memory against the already-fetched rows (a `List of [Code[20]]` membership check) rather than pushing the grouping into the query itself, which is a reasonable follow-up if that list ever needs to scale further (see Section 11).
- **Matches the project's own stated convention.** Record-based filtering with an OR-list is a valid alternative and was considered; Query was the specific instruction for this rewrite (single-pass, no per-row Record round trip semantics) and is the more idiomatic AL choice for a procedure whose entire job is "flatten some rows into JSON," as opposed to a procedure that also needs to write back to those rows.

5.3 Where the Query object sits in the call chain

The Query is not called directly by the background task, nor does it appear anywhere on page 50620 — it sits one layer down, as an implementation detail local to the two codeunit 50613 procedures that need it. Tracing the full chain from the user's click down to the SQL statement the Query compiles to:

1. **Page 50620** (parent session) — e.g. `OnShowResourcesForTask` → `EnqueueFilteredResourcePanelReload` → `CurrPage.EnqueueBackgroundTask(..., Codeunit::"Gantt BG Resource Panel Data", ...)`.
2. **Codeunit 50713** "Gantt BG Resource Panel Data" (child session) — `OnRun()` reads the Dictionary, then calls `GanttChartDataHandler.GetDayPlanningsByJobTaskAsJson(JobTaskKeys, FromDate, ToDate)` and `...GetResourcesByJobTaskAsJson(...)`.
3. **Codeunit 50613** "GanttChartDataHandler" — *inside* those two procedures, a local variable `DayPlanningByJobTaskQry: Query "Day Planning By Job Task"` is declared, filtered (`SetFilter` on `JobNoFilter` / `JobTaskNoFilter` / `PlanDateFilter`), then driven with `Open()` / `while Read()` / `Close()` — this is the only place in the whole codebase the Query object is actually opened.
4. **Query 50712** "Day Planning By Job Task" — compiles its one `dataitem` and column list into a single SQL `SELECT` against the "Day Planning" table with the three filters applied as a `WHERE` clause; the database returns all matching rows in one response.

So structurally: the Query object is a *data-access primitive*, owned by and private to codeunit 50613 (the same codeunit that owned the old `DayPlanning.FindSet()` loop it replaces); codeunit 50713 (the background task) is a *caller* that never sees the Query directly, only the JSON text the codeunit 50613 procedures hand back; and page 50620 is two layers further removed still, only ever seeing the fully-built JSON arrive via `OnPollResourcePanelResult` (Section 7.1–7.2). This layering is deliberate: it is the same reason the two codeunit 50613 procedures' public signatures didn't need to change in a way that leaked Query internals — they still just take a scope (now `List of [Text]` keys instead of a marked Record) and a date range, and return `Text`, exactly as before.

`AFTER — codeunit_50613_GanttChartDataHandler.al, GetDayPlanningsByJobTaskAsJson`

```
procedure GetDayPlanningsByJobTaskAsJson(JobTaskKeys: List of [Text]; FromDate: Date; ToDate: Date) JsonText:
Text
var
    DayPlanningByJobTaskQry: Query "Day Planning By Job Task";
    JsonArray: JsonArray;
    JobNoFilterText: Text;
    JobTaskNoFilterText: Text;
    PairSet: List of [Text];
begin
    if JobTaskKeys.Count() = 0 then begin
        JsonArray.WriteTo(JsonText);
        exit;
    end;

    BuildJobTaskKeyFilters(JobTaskKeys, JobNoFilterText, JobTaskNoFilterText, PairSet);
    DayPlanningByJobTaskQry.SetFilter(JobNoFilter, JobNoFilterText);
    DayPlanningByJobTaskQry.SetFilter(JobTaskNoFilter, JobTaskNoFilterText);
    ApplyPlanDateFilter(DayPlanningByJobTaskQry, FromDate, ToDate);

    if DayPlanningByJobTaskQry.Open() then begin
        while DayPlanningByJobTaskQry.Read() do
            // The two OR-lists (all distinct Job Nos | all distinct Job Task Nos across
            // the requested keys) are independent, so a row could satisfy both without
            // being one of the actual requested pairs - re-check in memory, no extra DB hit.
            if PairSet.Contains(DayPlanningByJobTaskQry.JobNo + '|' + DayPlanningByJobTaskQry.JobTaskNo) then
                JsonArray.Add(BuildDayPlanningJsonObject(/* 17 query columns -> same JSON shape as before */));
        DayPlanningByJobTaskQry.Close();
    end;

    JsonArray.WriteTo(JsonText);
end;
```

The result: one `Query.Open()` / `Read()` pass regardless of how many Job Tasks are in scope, instead of one `FindSet()` per task. `GetResourcesByJobTaskAsJson` was rewritten the same way, and its distinct-resource collection (previously a second per-task `FindSet()` loop) now happens in memory against the rows already fetched by the same query pass, using a `List of [Code[20]]` membership check.

> **EXPLICITLY OUT OF SCOPE**
>
> `GetJobTasksAsJson` was left untouched — it already runs as a single merge-joined pass over two independently filtered cursors (not the per-row bottleneck this change targets), and the user's request was for Job Task loading to stay direct/synchronous regardless. Codeunit 50615 "Gantt Update Data" (drag/drop persistence) is unrelated and was not touched.

6. Implementation Detail — Codeunit 50713 (Background Task Target)

A page background task codeunit is a plain codeunit (not an interface implementation) whose `OnRun()` trigger is invoked without a record, cannot render UI, and can only read the database — an attempted write raises a permission error. It reads its input via `Page.GetBackgroundParameters()` and returns results via `Page.SetBackgroundTaskResult()`.

`NEW — src/dhx/ganttdemo2/codeunit_50713_GanttBGResourcePanelData.al` **NEW OBJECT**

```
codeunit 50713 "Gantt BG Resource Panel Data"
{
    trigger OnRun()
    var
        GanttChartDataHandler: Codeunit "GanttChartDataHandler";
        TaskParameters: Dictionary of [Text, Text];
        Result: Dictionary of [Text, Text];
        Mode: Text;
        LoadResources: Boolean;
        LoadDayPlannings: Boolean;
        ResourcesJson: Text;
        DayPlanningsJson: Text;
        JobTaskKeys: List of [Text];
        FromDate: Date;
        ToDate: Date;
    begin
        TaskParameters := Page.GetBackgroundParameters();
        Mode := GetParam(TaskParameters, 'Mode');
        LoadResources := GetParam(TaskParameters, 'LoadResources') = 'true';
        LoadDayPlannings := GetParam(TaskParameters, 'LoadDayPlannings') = 'true';

        case Mode of
            'Filtered':
                begin
                    JobTaskKeys := GetParam(TaskParameters, 'JobTaskKeys').Split(';');
                    EvaluateDateParam(TaskParameters, 'FromDate', FromDate);
                    EvaluateDateParam(TaskParameters, 'ToDate', ToDate);
                    if LoadResources then
                        ResourcesJson := GanttChartDataHandler.GetResourcesByJobTaskAsJson(JobTaskKeys,
FromDate, ToDate);
                    if LoadDayPlannings then
                        DayPlanningsJson := GanttChartDataHandler.GetDayPlanningsByJobTaskAsJson(JobTaskKeys,
FromDate, ToDate);
                end;
            'Default':
                begin
                    // ... GetResourcesAsJson() / GetDayPlanningsAsJson(AnchorDate, JobFilter, JobTaskFilter)
                end;
        end;

        Result.Add('resourcesJson', ResourcesJson);
        Result.Add('dayPlanningsJson', DayPlanningsJson);
        Page.SetBackgroundTaskResult(Result);
    end;
}
```

Two scopes are supported via one `Mode` parameter — `Filtered` (one task + its children, right-click / post-drag reload) and `Default` (the unfiltered panel) — and either half (`LoadResources` / `LoadDayPlannings`) can be requested independently, so a caller that only needs one (e.g. `ShowResourcePanel` only wants Resources) doesn't pay to build the other.

7. Platform Finding: Control Add-in Callbacks Are Disallowed in Completion Triggers

> **LIVE-TESTED FINDING, CONTRADICTS THE GENERAL MICROSOFT LEARN GUIDANCE**
>
> Microsoft's documentation states that a page background task's callback triggers "can't execute UI operations, except notifications and control updates" — implying a control add-in method call from `OnPageBackgroundTaskCompleted` is a

> permitted "control update." Live testing against this environment's BC Server showed otherwise: any `CurrPage.DHXGanttControl2.*` call issued directly from `OnPageBackgroundTaskCompleted` (or `OnPageBackgroundTaskError`) is rejected with an error to the effect of *"attempted to issue a client callback on an Automation object... disallowed callback was issued from the OnPageBackgroundTaskCompleted or OnPageBackgroundTaskError trigger."*

7.1 Workaround: stash-and-poll

The completion trigger only stores the result in plain AL page variables and flips a flag — no control add-in call:

`page_50620_GanttDemo.al, OnPageBackgroundTaskCompleted`

```
trigger OnPageBackgroundTaskCompleted(TaskId: Integer; Results: Dictionary of [Text, Text])
begin
    if TaskId <> ResourcePanelTaskId then
        exit;   // superseded by a later reload - discard

    // Does NOT call CurrPage.DHXGanttControl2.* here (see Section 7).
    if Results.ContainsKey('resourcesJson') then
        PendingResourcesJson := Results.Get('resourcesJson');
    if Results.ContainsKey('dayPlanningsJson') then
        PendingDayPlanningsJson := Results.Get('dayPlanningsJson');
    PendingResultAvailable := true;
end;
```

Delivery into the control add-in happens later, from a *normal, JS-initiated synchronous trigger call* — the same call shape every other `CurrPage.DHXGanttControl2.*` call on this page already uses successfully:

```
trigger OnPollResourcePanelResult()
begin
    if not PendingResultAvailable then
        exit;

    PendingResultAvailable := false;
    if PendingResourcesJson <> '' then
        CurrPage.DHXGanttControl2.LoadResourcesData(PendingResourcesJson);
    if PendingDayPlanningsJson <> '' then
        CurrPage.DHXGanttControl2.LoadDayPlanningsData(PendingDayPlanningsJson);
    Clear(PendingResourcesJson);
    Clear(PendingDayPlanningsJson);
    CurrPage.DHXGanttControl2.StopResourcePanelPolling();
end;
```

7.2 What drives the poll: a bounded client-side timer

Two small additions to the control add-in interface (`GantttControl.controladdin.al`) connect the two triggers above to JavaScript:

```
event OnPollResourcePanelResult();
procedure NotifyResourcePanelTaskPending();  // arms the poll loop, called right after enqueue
procedure StopResourcePanelPolling();        // called once OnPollResourcePanelResult actually delivered data
```

and in `wrapper.js`, a short interval timer (500 ms per tick, capped at 60 ticks ≈ 30 s) that self-stops once data arrives or the cap is hit — so an idle Gantt page is never polled indefinitely:

```
var RESOURCE_PANEL_POLL_INTERVAL_MS = 500;
var RESOURCE_PANEL_POLL_MAX_ATTEMPTS = 60; // 60 x 500ms = 30s generous ceiling

function NotifyResourcePanelTaskPending() {
  if (_resourcePanelPollTimer) clearInterval(_resourcePanelPollTimer);
  _resourcePanelPollAttempts = 0;
  _resourcePanelPollTimer = setInterval(function () {
    _resourcePanelPollAttempts++;
    if (_resourcePanelPollAttempts > RESOURCE_PANEL_POLL_MAX_ATTEMPTS) {
      clearInterval(_resourcePanelPollTimer);
      _resourcePanelPollTimer = null;
      return;
    }
    Microsoft.Dynamics.NAV.InvokeExtensibilityMethod("OnPollResourcePanelResult", []);
  }, RESOURCE_PANEL_POLL_INTERVAL_MS);
}

function StopResourcePanelPolling() {
  if (_resourcePanelPollTimer) { clearInterval(_resourcePanelPollTimer); _resourcePanelPollTimer = null; }
}
```

> **WHY THIS IS STILL FULLY ASYNCHRONOUS**
>
> The poll is a client-side JavaScript timer, not a server-side wait. The BC session issuing the enqueue trigger returns immediately in every case (Section 3.2, step 2); nothing on the server blocks waiting for the background task. The polling only decides *when* the already-computed result gets handed to the control add-in, at 500 ms granularity, capped so it cannot run forever.

7.3 Stale-result guard

`ResourcePanelTaskId` always holds the TaskId of the *most recently enqueued* background task. Both `OnPageBackgroundTaskCompleted` and `OnPageBackgroundTaskError` check `TaskId <> ResourcePanelTaskId` and discard the result if a newer reload has since been enqueued (e.g. the user clicked Previous then Next again before the first task returned) — verified live under rapid navigation with no incorrect data ending up in the panel.

7.4 Secondary fix: `Format(Boolean)` is localized

> **BUG FOUND DURING IMPLEMENTATION**
>
> `Format(TrueOrFalseBoolean)` in AL returns the localized caption ("Yes"/"No"), not the literal text `"true"` / `"false"`. Codeunit 50713's `= 'true'` parameter checks were silently always false until this was caught and replaced with a literal helper.

```
local procedure BoolToParamText(Value: Boolean): Text
begin
    if Value then
        exit('true');
    exit('false');
end;
```

8. Verification

8.1 Correctness — Query output vs. live BC data

Right-clicked "Show Job Resources" on a real task (Job `10000` / Job Task `1080`, period 2026-09-08–2026-09-10). The JS-side result for that task contained 8 Day Planning rows. The same window was independently queried directly against Business Central using the `List_DayPlannings_PAG50606` API. Result: **exact match** on all 8 SystemIds, dates, hours, resource assignment, plan status, and Work Order No. — confirming the Query-based rewrite (Section 5) preserves the original data semantics.

8.2 Behavior & timing — live browser (Playwright, Chromium, signed-in web client)

Scenario	Result
Initial open	Task bars render immediately; resource panel hidden; **zero** Day Planning rows fetched during this phase (table: 28,025 rows total). Measured ~3.5 s from control add-in bootstrap to "Gantt render completed" (isolates add-in/AL work from outer BC shell & auth).
Show Resource Panel	Panel opens immediately, populates shortly after (async); poll auto-stopped after 1 attempt once data arrived.
Right-click "Show Job Resources"	Filters correctly to the clicked task + children (see 8.1 for the exact data cross-check).
Rapid Previous/Next	No errors, no crash; stale-result guard (7.3) kept the final panel state consistent with the last click.
Drag-refresh	Reuses the identical, already-validated enqueue→poll→deliver path used above (not independently drag-simulated).
Browser console	No new errors introduced — only pre-existing baseline noise unrelated to this change.
`al_compile` / `al_build`	Clean — no errors, no new warnings.

> **NO FORMAL BEFORE/AFTER BENCHMARK**
>
> A side-by-side timing comparison against the pre-change synchronous code was attempted (via `git stash`) but abandoned: this working directory is OneDrive-synced, and the stash operation raced with OneDrive's own file sync rather than reliably landing on disk. Reverting further to force a clean A/B was judged not worth the risk to the working tree, and the qualitative result (0 rows fetched vs. a full date-window slice, previously) was accepted as sufficient evidence of the fix.

9. Post-Deployment Fix: Show Resource Panel Omitted Day Plannings

Found after the initial rollout, via live user testing rather than the original verification pass (Section 8): clicking **Show Resource Panel** without first right-clicking a task bar left the panel showing the correct resource list but every resource at **0h** Total Hours, with no Day Planning bars — indefinitely, not as a transient loading state.

9.1 Root cause

The `ShowResourcePanel` action's `OnAction` trigger enqueued a background task requesting Resources only:

`BEFORE (the bug) — page_50620_GanttDemo.al, action(ShowResourcePanel)`

```
trigger OnAction()
begin
    ResourcePanelFlag := true;
    CurrPage.DHXGanttControl2.SetResourcePanelVisibility(true);
    ClearResourcePanelFilter();
    EnqueueDefaultResourcePanelReload(true, false);   // ← pLoadDayPlannings = false
end;
```

This mirrored the *pre-refactor* synchronous code, which also only called `LoadResourceData()` here — but that old code could get away with it because `ControlReady` used to eagerly load **all** Day Plannings for the visible window synchronously on every page open (the very blocking behavior Sections 2–4 of this document removed). By the time a user clicked "Show Resource Panel" under the old code, Day Planning data was already sitting in the DHTMLX Gantt's client-side store from that eager load; only the resource list needed a fresh fetch. Once the async migration removed that eager load from `ControlReady` (deliberately — Section 4.1), the client-side store is genuinely empty of Day Planning data until something explicitly requests it, and `ShowResourcePanel` was the one call site never updated to request it. The adjacent `OnResetResourceFilter` trigger (Section 4, a few lines above `ShowResourcePanel` in the same file) already had the correct call and was unaffected.

9.2 Fix

`AFTER — page_50620_GanttDemo.al, action(ShowResourcePanel)`

```
trigger OnAction()
begin
    ResourcePanelFlag := true;
    CurrPage.DHXGanttControl2.SetResourcePanelVisibility(true);
    ClearResourcePanelFilter();
    EnqueueDefaultResourcePanelReload(true, true);    // ← now requests Day Plannings too
end;
```

One argument changed, matching the pattern already used by `OnResetResourceFilter`. No change to the enqueue/poll/completion-trigger mechanism itself (Sections 3–7) — that machinery was already correct; this call site was simply asking it for the wrong half of the data.

9.3 Live verification

Scenario	Result

Exact reported repro: open Gantt fresh, do *not* click any task bar, go straight to Related → Show Resource Panel	Panel opens immediately and populates with real data shortly after: actual Total Hours per resource (e.g. Arnoud Wolthuis 272h, Assia Lammerts 212h) and visible day-planning hour bars across the timeline — no more 0h placeholders.
Refresh Data, clicked after the panel was already open	Panel stays correctly populated; no regression.
Right-click "Show Job Resources" on a task bar (the already-working `Filtered` path)	Unaffected — still correct (one narrow single-day task legitimately showed 0h, i.e. no Day Plannings recorded for that day, not a bug; a wider-window task showed real populated hours).

> **WHY THIS SLIPPED PAST THE ORIGINAL VERIFICATION PASS**
>
> Section 8's live verification exercised Show Resource Panel, but its check (Section 8.2) confirmed the panel populated *asynchronously* (poll auto-stopped once data arrived) without independently re-confirming the delivered data actually included non-zero Day Planning hours for the no-prior-task-click case — the async delivery mechanism was working exactly as designed, it was just being asked to deliver an incomplete dataset. A useful lesson for verifying async UI: confirming a load "completed" is not the same as confirming what it loaded was correct/complete.

10. Object Inventory

Object	Type	Status	Role
`Page 50620 "Gantt Demo DHX 2"`	Page	MODIFIED	ControlReady/LoadAllData restructured; new enqueue helpers; new OnPageBackgroundTaskCompleted/Error/OnPollResourcePanelResult triggers; post-deployment fix to `ShowResourcePanel` (Section 9)
`Codeunit 50613 "GanttChartDataHandler"`	Codeunit	MODIFIED	`GetDayPlanningsByJobTaskAsJson` / `GetResourcesByJobTaskAsJson` rewritten on Query 50712; `GetJobTasksAsJson` untouched
`"DHX Gantt Control 2"`	Control Add-in	MODIFIED	+1 event (`OnPollResourcePanelResult`), +2 procedures (`NotifyResourcePanelTaskPending` , `StopResourcePanelPolling`)
`wrapper.js`	Add-in script	MODIFIED	Bounded client-side poll loop (Section 7.2)
`Query 50712 "Day Planning By Job Task"`	Query	NEW	Single-pass replacement for the per-task `FindSet()` loop
`Codeunit 50713 "Gantt BG Resource Panel Data"`	Codeunit	NEW	Page Background Task target — Filtered/Default modes
`Codeunit 50615 "Gantt Update Data"`	Codeunit	unchanged	Out of scope — unrelated (drag/drop persistence only)

11. Known Limitations & Follow-ups

- **No isolated before/after benchmark exists** (Section 8.2) — obtaining one safely would require an isolated git worktree outside the OneDrive-synced project folder; not pursued as the qualitative evidence (0 rows fetched vs. a full slice) was judged sufficient.
- **Poll ceiling is 30 s.** If a background task legitimately takes longer than that (very large scope, server under load), the client stops polling before the result exists in the completion trigger's stash; the user would need to trigger another reload. The background task itself is separately capped at a 30,000 ms server-side timeout in the `EnqueueBackgroundTask` call, so the two limits are aligned.
- **Default concurrency limit.** Business Central allows 5 concurrent page background tasks per parent session by default; not a concern for this single-panel use case, but relevant if more background tasks are added to this page later.
- **Drag-refresh path reuses, but was not independently, live-tested** with a real drag simulation — it shares the same enqueue/poll/deliver code path validated by the other scenarios.

Appendix A — Page Background Task API Reference

For reference when reading or extending this code (source: Microsoft Learn, "Page Background Tasks"):

Element	Signature / Note
`CurrPage.EnqueueBackgroundTask`	`(var TaskId: Integer; CodeunitId: Integer; [Parameters: Dictionary of [Text, Text]], [Timeout: Integer], [ErrorLevel: PageBackgroundTaskErrorLevel])`
`Page.GetBackgroundParameters()`	Called inside the background codeunit's `OnRun()` to read the input dictionary
`Page.SetBackgroundTaskResult(Result)`	Called inside `OnRun()` to return the output dictionary
`OnPageBackgroundTaskCompleted(TaskId: Integer; Results: Dictionary of [Text, Text])`	Page trigger. Callback triggers cannot render UI except notifications — and, per this document's Section 7, **not** control add-in calls either, despite general docs implying otherwise
`OnPageBackgroundTaskError(TaskId: Integer; ErrorCode: Text; ErrorText: Text; ErrorCallStack: Text; var IsHandled: Boolean)`	Page trigger, fired on failure/timeout of the background task
Session characteristics	Read-only (a write raises a permission error); own `OpenCompany` /transaction; canceled if the page closes; default 5 concurrent tasks per parent session; max 600,000 ms timeout

Appendix B — Glossary of Terms

Term	Meaning in this document
Parent session	The normal client/server connection for an open BC page — runs every trigger the user directly interacts with (button clicks, control add-in events, `ControlReady`).
Child session	A separate, temporary server connection Business Central creates to run a Page Background Task codeunit. Own `OpenCompany` , own transaction, read-only, ends when the codeunit's `OnRun()` finishes.

Page Background Task	The BC platform feature (available since 2019 release wave 2) used here: `CurrPage.EnqueueBackgroundTask` schedules a codeunit to run in a child session; `OnPageBackgroundTaskCompleted` / `OnPageBackgroundTaskError` report the outcome back to the parent session.
TaskId	An `Integer` the platform assigns when `EnqueueBackgroundTask` is called (via its `var TaskId` parameter). Used to match a later completion/error callback to the specific enqueue call that caused it — essential once more than one task can be in flight (Section 7.3).
`Dictionary of [Text, Text]`	The only data shape allowed to cross the parent/child session boundary as background-task input or output — see Section 3.3.
Round trip	One request/response exchange between AL and the underlying SQL database. Each separate `Record.FindSet()` call is (at least) one round trip; the goal of both the async migration and the Query rewrite is to minimize how many of these happen, and when.
N+1 (query) problem	A pattern where fetching a list of N parent rows, then querying once per parent row for its children, produces N+1 total round trips instead of 1 or 2. See Section 5 for the specific instance fixed here.
Query object	An AL object type (`query 50712 "Day Planning By Job Task"` here) that declares a fixed, filterable, projected set of columns over one or more tables and compiles to a single SQL statement, opened/read/closed like a lightweight read-only cursor. See Sections 5.2–5.3.
`Record.Mark()`	An AL mechanism for flagging a subset of rows within one Record variable's result set as "selected," used across multiple loops without re-filtering. Session-local — does not survive into a background task's child session, which is why this migration replaced marked-record scopes with plain `"JobNo|JobTaskNo"` text keys wherever a background task needed to receive that scope (Section 3.3).
Poll / polling	Here: a bounded client-side JavaScript timer (wrapper.js, Section 7.2) that repeatedly asks AL "is a result ready yet?" via a normal synchronous trigger call, until it gets one or hits its attempt cap. Distinct from the background task itself being asynchronous — the poll only governs when an already-finished result gets delivered to the UI.
Stale-result guard	The `TaskId <> ResourcePanelTaskId` check in `OnPageBackgroundTaskCompleted` / `OnPageBackgroundTaskError` (Section 7.3) that discards a background task's result if a newer request has since superseded it.
Control add-in callback	Any `CurrPage.<AddInName>.SomeMethod(...)` call — a server-to-client instruction telling the browser-hosted JavaScript control to do something. Confirmed (Section 7) to be disallowed specifically from inside `OnPageBackgroundTaskCompleted` / `OnPageBackgroundTaskError`, though allowed from ordinary triggers.

Appendix C — How to Verify This Yourself

Every code excerpt in this document was copied directly from the current working tree at the time of writing (not reconstructed from memory), so you can open these files side by side with this document and confirm each claim first-hand. Suggested reading order, using your editor's search (Ctrl+F) for each term:

1. Open `src/dhx/ganttdemo2/page_50620_GanttDemo.al` and search for `trigger ControlReady` — confirm `ResourcePanelFlag := false` now appears *before* the call to `LoadAllData()` (Section 4.1).
2. In the same file, search for `CurrPage.EnqueueBackgroundTask` — there are exactly two matches, inside `EnqueueFilteredResourcePanelReload` and `EnqueueDefaultResourcePanelReload` (Sections 3.2/4.2). Confirm nothing in either surrounding procedure waits on the call's result before returning.
3. Search the same file for `trigger OnPageBackgroundTaskCompleted` and `trigger OnPollResourcePanelResult` — confirm the first never references `CurrPage.DHXGanttControl2`, and the second is the only trigger that does, for this data (Section 7.1).

4. Open `src/query/query_50712_DayPlanningByJobTask.al` in full — it is short (Section 5); confirm every column it declares is one `BuildDayPlanningJsonObject` (codeunit 50613) actually reads.
5. Open `src/dhx/ganttdemo2/codeunit_50613_GanttChartDataHandler.al` and search for `GetDayPlanningsByJobTaskAsJson` — confirm its parameter list now reads `JobTaskKeys: List of [Text]`, not a Record, and that its body opens `Query "Day Planning By Job Task"` exactly once, outside any per-task loop (Sections 5.1, 5.3).
6. Open `src/dhx/ganttdemo2/codeunit_50713_GanttBGResourcePanelData.al` in full (it is one short file, Section 6) — confirm it never calls anything on `CurrPage`, consistent with running in a separate, UI-less child session.
7. Open `src/dhx/ganttdemo2/GantttControl.controladdin.al` and search for `OnPollResourcePanelResult`, `NotifyResourcePanelTaskPending`, `StopResourcePanelPolling` — confirm these three additions are the only control add-in interface changes this enhancement made (Section 7.2).
8. Open `src/dhx/ganttdemo2/wrapper.js` and search for `RESOURCE_PANEL_POLL_MAX_ATTEMPTS` — confirm the poll is bounded (currently 60 × 500 ms), not indefinite.
9. To see the platform finding from Section 7 for yourself, try calling any `CurrPage.DHXGanttControl2.*` method directly from `OnPageBackgroundTaskCompleted` in a scratch build — BC Server will reject it with the error text quoted in Section 7.
